

AIM: Implement and analyze various methods of studying correlation and regression including Karl Pearson's coefficient and simple linear regression.

Theory

1.What is correlation? How is it different from regression?

Correlation is a statistical measure that shows how strongly two variables are related and in which direction they move. Its value ranges from -1 to +1. A value near +1 means a strong positive relationship, while a value near -1 means a strong negative relationship. However, correlation only shows association and does not prove that one variable causes the other.

Regression, on the other hand, goes a step further. It not only shows the relationship but also helps in predicting one variable based on another. It provides a mathematical equation that explains how the dependent variable changes when the independent variable changes.

2. What is simple linear regression? Explain with suitable example?

Simple linear regression is a method used to describe the relationship between one independent variable (X) and one dependent variable (Y) using a straight line. The equation is written as: $Y = a + bX$, where a is the intercept and b is the slope of the line.

For example, if we want to understand how the number of hours studied affects exam scores, we can use simple linear regression. If the slope is positive, it means that as study time increases, marks also increase.

3.What is multicollinearity? Does it affect simple regression?

Multicollinearity happens when two or more independent variables in a regression model are highly related to each other. This creates problems in identifying the individual effect of each variable and can make the results unreliable.

In simple linear regression, there is only one independent variable. Because of this, multicollinearity cannot occur and does not affect simple regression.

4.How do you calculate Pearson correlation in Python?

Pearson correlation in Python can be calculated using libraries like NumPy or Pandas. With NumPy, you can use `np.corrcoef(x, y)` where x and y are the datasets. In Pandas, if your data is stored in a DataFrame, you can use `df['x'].corr(df['y'])`.

Both methods return a value between -1 and +1, showing how strong and in which direction the linear relationship exists between the variables.

5.A dataset shows strong correlation but poor prediction accuracy. Why?

A strong correlation does not always mean that predictions will be accurate. Correlation only measures how closely two variables move together in a linear way. Poor prediction can happen due to reasons like outliers, non-linear patterns, missing important variables, or overfitting.

So even if two variables are strongly related, the model might still fail to make accurate predictions if it does not capture the true pattern in the data.

```
import numpy as np

# Sample data
X = [100, 200, 300, 400, 500]
Y = [15, 25, 35, 45, 55]

# Manual implementation of Pearson correlation
def pearson_coefficient(x, y):
    n = len(x)
    mean_x = np.mean(x)
    mean_y = np.mean(y)

    numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
    denominator = (sum((x[i] - mean_x)**2 for i in range(n)) * sum((y[i] - mean_y)**2 for i in range(n)))**0.5

    return numerator / denominator

# Calculate Karl Pearson's correlation coefficient
r = pearson_coefficient(X, Y)
print(f"Karl Pearson's Correlation Coefficient (manual): {r:.4f}")

# Verification using scipy
from scipy.stats import pearsonr
r_scipy, _ = pearsonr(X, Y)
print(f"Verified using scipy: {r_scipy:.4f}")
```

```
Karl Pearson's Correlation Coefficient (manual): 1.0000
Verified using scipy: 1.0000
```

```

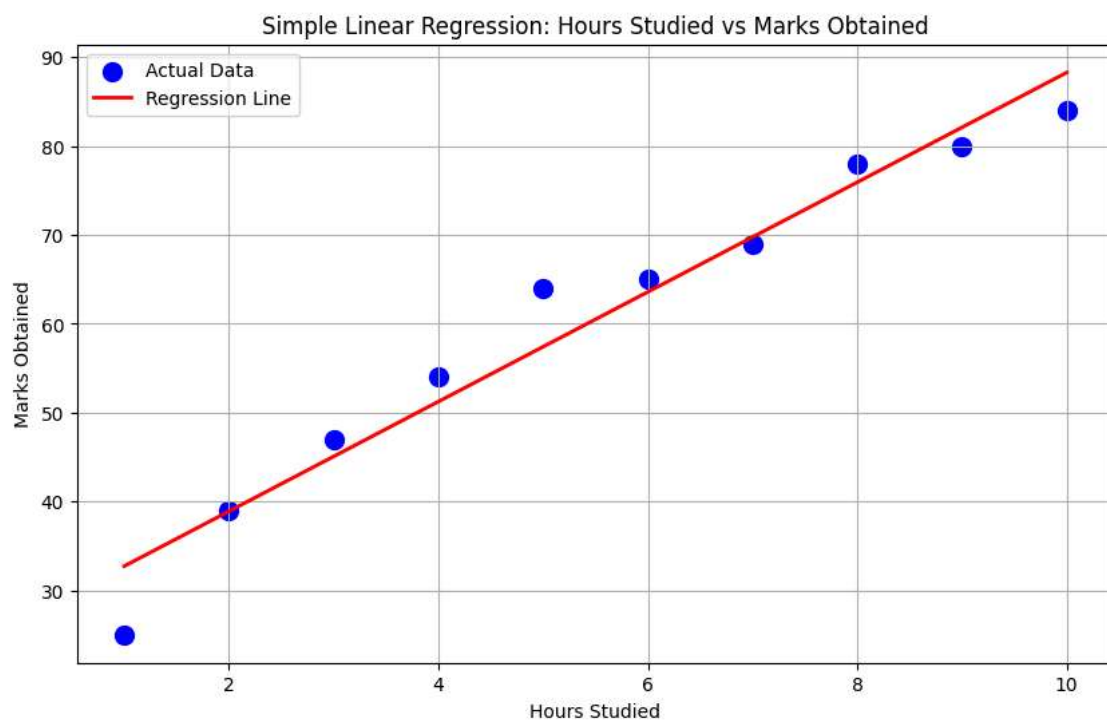
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Step 1: Updated Sample Data
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks_Obtained': [25, 39, 47, 54, 64, 65, 69, 78, 80, 84]
}

df = pd.DataFrame(data)
# Step 2: Split data into input (X) and output (y)
X = df[['Hours_Studied']] # Independent variable
y = df[['Marks_Obtained']] # Dependent variable
# Step 3: Train the Linear Regression Model
model = LinearRegression()
model.fit(X, y)
# Step 4: Get model parameters
slope = model.coef_[0]
intercept = model.intercept_
print(f"Regression Equation: Marks = {slope:.2f} * Hours + {intercept:.2f}")
# Step 5: Predict values
y_pred = model.predict(X)
# Step 6: Evaluate the Model
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print(f" Mean Squared Error (MSE): {mse:.2f}")
print(f" R² Score: {r2:.4f}")
# Step 7: Visualize the regression line
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Actual Data', s=100)
plt.plot(X, y_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Simple Linear Regression: Hours Studied vs Marks Obtained')
plt.xlabel('Hours Studied')
plt.ylabel('Marks Obtained')
plt.grid(True)
plt.legend()
plt.show()

```

Regression Equation: Marks = 6.18 * Hours + 26.53
Mean Squared Error (MSE): 14.40
R² Score: 0.9563



Conclusion

The simple linear regression model indicates a strong positive connection between the number of hours studied and the marks scored. As the time spent studying increases, the marks also tend to rise, which is shown by the upward trend of the regression line. The equation of the model helps in estimating marks based on study hours.

Performance measures like Mean Squared Error (MSE) and R² score help us understand how well the model represents the data. A smaller MSE and a higher R² value mean the model fits the data well and explains most of the variation in marks.

In conclusion, the analysis shows that study time plays an important role in academic results, and linear regression is a useful tool for understanding and predicting this relationship.